SGDD

Sistema de Gestion de Dulcerias

DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD)

Version 1.0.0 | 2026

Autor: Bryan Stiven Bonilla Rosas

Plataforma SaaS Multi-Tenant

INFORMACION DEL DOCUMENTO

Campo	Detalle
Titulo del Documento	Documento de Arquitectura de Software (SAD)
Sistema	SGDD - Sistema de Gestion de Dulcerias
Clasificacion	Plataforma SaaS Multi-Tenant
Version	1.0.0
Autor	Bryan Stiven Bonilla Rosas
Fecha	2026
Estado	Activo - En Produccion
Estandar de referencia	ISO/IEC 42010 - Descripcion de Arquitecturas de Software

Historial de Versiones

Version	Fecha	Autor	Descripcion del Cambio
1.0.0	2026	Bryan Stiven Bonilla Rosas	Creacion inicial del documento SAD

1. INTRODUCCION

Este documento describe la arquitectura del sistema SGDD (Sistema de Gestion de Dulcerias), una plataforma SaaS disenada para gestionar operaciones comerciales de dulcerias. El presente SAD tiene como objetivo proporcionar una vision clara, detallada y trazable de la arquitectura del sistema, cubriendo decisiones de diseno, estructura de componentes, patrones utilizados e interacciones entre servicios.

1.1 Proposito

El proposito de este documento es:

- Describir la arquitectura del sistema SGDD a nivel estructural, logico y fisico.
- Documentar las decisiones arquitectonicas tomadas durante el diseno del sistema.
- Servir como referencia tecnica para el equipo de desarrollo, QA e infraestructura.
- Garantizar la trazabilidad entre requisitos, diseno e implementacion.

1.2 Alcance

SGDD es una plataforma SaaS que permite a multiples dulcerias gestionar sus operaciones de forma centralizada bajo un modelo multi-tenant. El sistema abarca los siguientes modulos funcionales:

Modulo	Descripcion
Punto de Venta (POS)	Registro de ventas en tiempo real con soporte a multiples metodos de pago
Control de Inventario	Gestion de productos por lotes, alertas de stock y trazabilidad
Gestion de Clientes	Registro, historial y segmentacion de clientes
Reportes y Analitica	Dashboards interactivos, KPIs y exportacion de datos
Prediccion de Ventas	Modelos ML para forecast de demanda
Bot Conversacional	Consultas en lenguaje natural sobre datos del negocio

1.3 Audiencia

- Arquitectos de software
- Desarrolladores backend y frontend
- Equipo de DevOps e infraestructura
- QA Engineers
- Stakeholders tecnicos del proyecto

1.4 Estandar de referencia

Este documento se alinea con el estandar ISO/IEC 42010:2011, que define la descripcion de arquitecturas de sistemas y software mediante vistas, puntos de vista y decisiones arquitectonicas documentadas.

2. PRINCIPIOS DE ARQUITECTURA

El sistema fue disenado siguiendo un conjunto de principios fundamentales que guian todas las decisiones arquitectonicas:

Principio	Descripcion	Impacto
Clean Architecture	Separacion estricta de dominio, aplicacion, infraestructura y presentacion.	Mantenibilidad y testabilidad
Independencia de Framework	La logica de negocio no depende directamente de frameworks externos.	Portabilidad y durabilidad del codigo
Escalabilidad Horizontal	El sistema permite desplegar multiples instancias del backend simultaneamente.	Alta disponibilidad
Seguridad Zero Trust	Cada solicitud HTTP es autenticada y autorizada de forma independiente.	Seguridad perimetral
Multi-Tenancy	Aislamiento de datos por tenant mediante tenant_id en cada entidad.	Seguridad de datos
Observabilidad	El sistema expone metricas, logs y trazas para monitoreo continuo.	Operaciones y SRE
Diseno API-First	La API REST es el contrato principal entre frontend y backend.	Integracion desacoplada
Domain-Driven Design (DDD)	El modelo de negocio refleja el dominio real de una dulceria.	Alineacion negocio-tecnologia

3. VISTA GENERAL DE ARQUITECTURA

El sistema sigue una arquitectura cliente-servidor distribuida, organizada en capas claramente definidas. La comunicacion entre el frontend y el backend se realiza mediante HTTP REST API, con autenticacion basada en JWT.

3.1 Diagrama de arquitectura general

Arquitectura de alto nivel del sistema SGDD:

```
  [ Navegador / App Movil ]            |      HTTP REST API     (JWT Auth /
HTTPS)            |    [ Backend FastAPI ]            |   +---------+---------+
|          |          | [PostgreSQL] [Redis] [IA/ML]  Base Datos  Cache   Modelos
```

3.2 Componentes principales

Componente	Tecnologia	Rol en el sistema
Frontend SPA	React + TypeScript	Interfaz de usuario para todos los modulos
API REST	FastAPI (Python)	Gateway principal de acceso al backend
Base de datos	PostgreSQL	Persistencia relacional con aislamiento multi-tenant
Cache	Redis	Cache de sesiones, rate limiting y datos frecuentes
Modelos IA	Scikit-Learn / Prophet	Prediccion de demanda y analitica avanzada
Bot NLP	spaCy / Transformers	Procesamiento de lenguaje natural para chatbot
Monitoreo	Prometheus + Grafana	Observabilidad y alertas del sistema
Contenedores	Docker + Docker Compose	Portabilidad y orquestacion de servicios

4. ARQUITECTURA LOGICA

El backend del sistema implementa Clean Architecture con cuatro capas principales. Esta decision garantiza separacion de responsabilidades, alta cohesion y bajo acoplamiento.

4.1 Diagrama de capas

```
  +-------------------------------+   |    API / Presentacion         | <-
Endpoints REST, Schemas   +-------------------------------+              |
+-------------------------------+   |  Aplicacion (Casos de Uso)    | <-
Orquestacion de negocio   +-------------------------------+              |
+-------------------------------+   |  Dominio (Reglas de Negocio)  | <-
Entidades, Value Objects   +-------------------------------+              |
+-------------------------------+   |  Infraestructura              | <- DB,
Redis, IA, APIs externas   +-------------------------------+
```

4.2 Capa de Dominio

El nucleo del sistema. Contiene las entidades y reglas de negocio que representan el modelo real de una dulceria. No depende de ninguna tecnologia externa.

Entidad	Descripcion
Producto	Representa un articulo vendible. Incluye precio, categoria, unidad de medida y estado.
Venta	Transaccion comercial. Contiene fecha, total, metodo de pago y referencia al vendedor.
DetalleVenta	Linea de venta individual. Asocia un producto con cantidad y precio unitario.
Cliente	Persona o empresa compradora. Incluye historial, segmento y preferencias.
Inventario	Estado actual de stock por producto. Dispara alertas cuando alcanza el minimo.
LoteInventario	Entrada de mercancia. Incluye fecha de vencimiento, proveedor y cantidad.

4.3 Capa de Aplicacion (Casos de Uso)

Orquesta la logica de negocio invocando entidades del dominio y servicios de infraestructura. Cada caso de uso es una clase independiente.

Caso de Uso	Descripcion
RegistrarVentaUseCase	Valida stock, registra venta, descuenta inventario y genera recibo
ConsultarStockUseCase	Retorna el estado actual del inventario con alertas activas
GenerarReporteUseCase	Construye reportes de ventas por periodo, producto o vendedor
ProcesarPrediccionUseCase	Invoca el modelo ML y persiste el forecast de demanda

ConsultarBotUseCase	Procesa la consulta en lenguaje natural y retorna respuesta estructurada
GestionarClienteUseCase	Alta, modificacion y consulta de clientes con su historial

4.4 Capa de Infraestructura

Proporciona implementaciones concretas de los repositorios definidos en el dominio. Aqui residen los adaptadores a tecnologias externas.

- Repositorios SQLAlchemy (PostgreSQL)
- Adaptador de cache Redis
- Cliente de modelos ML (scikit-learn / Prophet)
- Servicio de notificaciones
- Proveedor de JWT

4.5 Capa de Presentacion

Exposicion de endpoints REST mediante FastAPI. Los schemas de entrada y salida se definen con Pydantic para validacion automatica.

```
POST   /api/v1/auth/login POST   /api/v1/ventas GET    /api/v1/ventas/{id} GET
/api/v1/inventario POST   /api/v1/inventario/lotes GET
/api/v1/reportes/ventas GET    /api/v1/predicciones POST
/api/v1/bot/consultar GET    /api/v1/clientes
```

5. ARQUITECTURA FISICA Y DESPLIEGUE

La arquitectura fisica esta basada en contenedores Docker orquestados con Docker Compose para entornos de desarrollo y produccion inicial. El sistema esta disenado para migracion futura a Kubernetes.

5.1 Servicios Docker

Servicio	Imagen Base	Puerto	Descripcion
backend	python:3.11-slim	8000	API FastAPI principal
frontend	node:18-alpine	3000	SPA React
postgres	postgres:15	5432	Base de datos relacional
redis	redis:7-alpine	6379	Cache y rate limiting
prometheus	prom/prometheus	9090	Recoleccion de metricas
grafana	grafana/grafana	3001	Visualizacion de metricas

5.2 Diagrama de despliegue

```
  docker-compose   |   +-- frontend (React)      :3000   |        |   |    HTTP
REST   |        |   +-- backend (FastAPI)     :8000   |        |     |   |
|     +-- redis  :6379   |        |   +-- postgres              :5432   +--
prometheus             :9090   +-- grafana                :3001
```

5.3 Estrategia de escalabilidad

Fase	Tecnologia	Descripcion
Actual	Docker Compose	Despliegue en un solo servidor o VPS
Escalado horizontal	Docker + Load Balancer	Multiples instancias del backend detras de Nginx/Traefik
Futuro	Kubernetes (K8s)	Orquestacion avanzada con auto-scaling
Cloud	AWS / Azure / GCP	Despliegue en proveedor cloud con servicios gestionados

6. ARQUITECTURA DE DATOS

El sistema utiliza PostgreSQL como base de datos relacional. El diseno de datos esta orientado a garantizar el aislamiento multi-tenant, la trazabilidad de operaciones y la eficiencia en consultas analiticas.

6.1 Modelo de datos principal

Tabla	Clave Primaria	tenant_id	Descripcion
tenants	id (UUID)	No aplica	Registro de cada dulceria en la plataforma
usuarios	id (UUID)	Si	Usuarios del sistema con roles y permisos
productos	id (UUID)	Si	Catalogo de productos por tenant
inventario	id (UUID)	Si	Stock actual por producto
lotes_inventario	id (UUID)	Si	Entradas de mercancia con fecha de vencimiento
ventas	id (UUID)	Si	Cabecera de transaccion de venta
detalle_ventas	id (UUID)	Si	Lineas de productos por venta
clientes	id (UUID)	Si	Registro de clientes por tenant
predicciones	id (UUID)	Si	Resultados de modelos ML por producto

6.2 Multi-Tenancy en datos

REGLA CRITICA: Toda consulta a la base de datos DEBE incluir la condicion WHERE tenant_id = :tenant_id. Este filtro es inyectado automaticamente por el ORM en cada operacion.

```
Tenant   |   +-- Usuarios         (tenant_id)   +-- Productos
(tenant_id)   |     |   |     +-- Inventario (tenant_id)   |     +-- Lotes
(tenant_id)   |   +-- Ventas           (tenant_id)   |     |   |     +--
DetalleVentas (tenant_id)   |   +-- Clientes         (tenant_id)   +--
Predicciones     (tenant_id)
```

6.3 Estrategia de indices

Tabla	Indices definidos
ventas	tenant_id, fecha, vendedor_id, cliente_id
productos	tenant_id, categoria, nombre

inventario	tenant_id, producto_id
detalle_ventas	venta_id, producto_id
usuarios	tenant_id, email (UNIQUE por tenant)

7. ARQUITECTURA DE SEGURIDAD

El sistema implementa un modelo Zero Trust donde cada solicitud es autenticada y autorizada de forma independiente, sin asumir confianza implicita por red o posicion en la infraestructura.

7.1 Mecanismos de seguridad

Mecanismo	Tecnologia	Descripcion
Autenticacion	JWT (RS256)	Token firmado con clave asimetrica. Expira en 60 minutos.
Hash de contrasenas	bcrypt (cost=12)	Las contrasenas nunca se almacenan en texto plano.
Control de acceso	RBAC	4 roles: Administrador, Vendedor, Operador, Supervisor.
Rate Limiting	Redis + SlowAPI	Maximo 100 req/min por IP. Bloqueo temporal ante ataques.
Aislamiento de datos	tenant_id obligatorio	Filtro en ORM que impide acceso cruzado entre tenants.
HTTPS	TLS 1.3	Toda comunicacion cifrada en transito.
Validacion de inputs	Pydantic v2	Validacion automatica de tipos, formatos y rangos.
Auditoria	Tabla audit_logs	Registro de operaciones criticas con usuario, fecha y accion.

7.2 Modelo RBAC

Rol	Permisos
Administrador	Acceso total al sistema, gestion de usuarios, configuracion del tenant y reportes avanzados.
Supervisor	Acceso a reportes, analitica, gestion de inventario y consulta de ventas.
Vendedor	Registro de ventas en POS, consulta de productos y atencion al cliente.
Operador	Gestion de inventario, recepcion de lotes y consulta de alertas de stock.

8. MODULO DE INTELIGENCIA ARTIFICIAL

El sistema integra capacidades de IA y ML para ofrecer funcionalidades de prediccion de demanda y bot conversacional de consultas.

8.1 Prediccion de demanda

Aspecto	Detalle
Objetivo	Predecir la demanda futura de productos para optimizar el inventario
Modelos utilizados	Prophet (Facebook) para series temporales con estacionalidad
Frecuencia de entrenamiento	Reentrenamiento semanal automatico con datos nuevos
Datos de entrada	Historial de ventas por producto, fecha, dia de semana y temporada
Salida	Forecast de ventas para los proximos 7, 14 y 30 dias por producto
Almacenamiento	Tabla predicciones con tenant_id, producto_id, fecha y cantidad estimada
Evaluacion	MAE, RMSE y coverage del intervalo de confianza al 95%

8.2 Bot conversacional

Aspecto	Detalle
Objetivo	Permitir consultas en lenguaje natural sobre datos del negocio
Tecnologia NLP	spaCy para extraccion de entidades + modelo de intenciones
Ejemplos de consultas	'Cuanto vendi ayer?' / 'Que productos tienen poco stock?' / 'Dame el reporte de la semana'
Flujo de procesamiento	Consulta -> NLP -> SQL Query generada -> Resultado formateado
Restricciones de seguridad	El bot solo puede leer datos del tenant autenticado. Sin acceso de escritura.

9. ARQUITECTURA DEVOPS Y CI/CD

El sistema utiliza contenedores Docker para garantizar portabilidad entre entornos y un pipeline de integracion continua para validar la calidad del codigo en cada cambio.

9.1 Pipeline CI/CD

```
  GitHub Repository        |   [Push / PR]        |   CI Pipeline   +----------
-+   | Lint      | (flake8, black, isort)   | Tests     | (pytest + coverage)
| Build     | (docker build)   | Validate  | (schema, migrations)   +----------
-+        |   [Merge a main]        |   Deploy automatico
```

9.2 Entornos

Entorno	Rama Git	Infraestructura	Proposito
Desarrollo	feature/*	Local Docker Compose	Desarrollo individual de funcionalidades
Staging	develop	VPS / Cloud staging	Integracion y pruebas de QA
Produccion	main	VPS / Cloud prod	Sistema en uso por clientes reales

9.3 Observabilidad

Herramienta	Funcion
Prometheus	Recoleccion de metricas del backend: latencia, errores, throughput
Grafana	Dashboards visuales con alertas configurables
Logs estructurados	JSON logs con nivel, timestamp, tenant_id y trace_id
Health endpoints	GET /health y GET /readiness para monitoreo de disponibilidad

10. DECISIONES ARQUITECTONICAS (ADR)

Las decisiones arquitectonicas del sistema se documentan mediante Architecture Decision Records (ADR). Cada ADR describe el contexto, la decision tomada y sus consecuencias.

ADR-001: Uso de FastAPI como framework backend

Campo	Detalle
Estado	Aceptado
Contexto	Se necesita un framework Python para construir la API REST con alta performance y documentacion automatica.
Decision	Se elige FastAPI por su soporte nativo a async/await, generacion automatica de OpenAPI y validacion con Pydantic.
Consecuencias	Alto rendimiento en operaciones I/O. Curva de aprendizaje moderada para el equipo.

ADR-002: Clean Architecture + Domain-Driven Design

Campo	Detalle
Estado	Aceptado
Contexto	El sistema tiene logica de negocio compleja que debe ser testeada independientemente de la infraestructura.
Decision	Implementar Clean Architecture con 4 capas y aplicar DDD para modelar el dominio de una dulceria.
Consecuencias	Mayor mantenibilidad y testabilidad. Mayor complejidad inicial de estructura de carpetas.

ADR-003: Multi-tenancy con tenant_id compartido

Campo	Detalle
Estado	Aceptado
Contexto	La plataforma debe soportar multiples dulcerias con aislamiento de datos.
Decision	Base de datos compartida con tenant_id como discriminador en cada tabla (Shared Schema).
Consecuencias	Menor costo operativo que bases separadas. Riesgo mitigado con filtros ORM obligatorios.

ADR-004: JWT para autenticacion

Campo	Detalle
Estado	Aceptado
Contexto	El sistema es stateless y necesita autenticacion sin sesiones en servidor.
Decision	JWT firmados con RS256, con expiracion de 60 minutos y refresh token de 7 dias.
Consecuencias	Arquitectura stateless escalable. Requiere estrategia de revocacion para tokens robados.

ADR-005: Redis para cache y rate limiting

Campo	Detalle
Estado	Aceptado
Contexto	Consultas frecuentes al inventario y reportes generan carga innecesaria en PostgreSQL.
Decision	Redis como cache distribuido con TTL configurable. Tambien usado para rate limiting con SlowAPI.
Consecuencias	Reduccion del 60-80% de consultas a DB en operaciones frecuentes. Complejidad adicional de invalidacion de cache.

11. RIESGOS TECNICOS Y MITIGACIONES

Riesgo	Probabilidad	Impacto	Estrategia de Mitigacion
Fuga de datos entre tenants	Baja	Critico	Filtros ORM obligatorios + pruebas de aislamiento en CI
Escalabilidad limitada en Docker Compose	Media	Alto	Arquitectura preparada para migracion a Kubernetes
Latencia alta en reportes complejos	Media	Medio	Cache Redis + indices optimizados + queries asincronas
Predicciones imprecisas del modelo ML	Media	Medio	Reentrenamiento continuo + intervalo de confianza visible
Perdida de datos por fallo en PostgreSQL	Baja	Critico	Backups diarios automatizados + replicacion en produccion
Ataques de fuerza bruta a la API	Alta	Alto	Rate limiting con Redis + bloqueo temporal de IPs
Downtime durante actualizaciones	Media	Alto	Blue-green deployments + health checks en CI/CD

12. ESTADO DOCUMENTAL DEL PROYECTO

El proyecto SGDD cuenta con documentacion tecnica completa alineada con estandares de ingenieria de software profesional:

Documento	Sigla	Descripcion	Estado
Especificacion de Requisitos de Software	SRS	Define todos los requisitos funcionales y no funcionales del sistema	Completo
Documento de Arquitectura de Software	SAD	Describe la arquitectura, componentes y decisiones de diseno (este documento)	Completo
Documento de Diseno de Software	SDD	Detalla el diseno detallado de modulos, clases y diagramas UML	Completo
Architecture Decision Records	ADR	Registro de decisiones arquitectonicas con contexto y justificacion	Completo
Plan de Pruebas de Software	STP	Estrategia de testing, casos de prueba y criterios de aceptacion	Completo

Este conjunto de documentos representa documentacion completa de ingenieria de software de nivel profesional, alineada con las mejores practicas de la industria para sistemas SaaS empresariales.